ИНСТРУКЦИЯ ПО ОБУЧЕНИЮ МОДЕЛИ

Сервис прогнозирования инцидентов в инженерных коллекторах

Задача №8 «Город»

Конкурс «Лидеры цифровых трансформаций 2026»

Постановщик: Департамент жилищно-коммунального хозяйства города Москвы

Эксплуатант: АО «Москоллектор»

Команда «Дискреция творца»

Москва, сентябрь 2026

Содержание

1. Что здесь обучается ... 3
2. Что потребуется ... 4
3. Обучение с нуля ... 5
- Шаг 1. Журнал в Parquet ... 5
- Шаг 2. Витрины ... 5
- Шаг 3. Тёзки и логические домены ... 5
- Шаг 4. Обучение ... 5
- Шаг 5. Кривая по горизонтам (необязательно) ... 6
- Шаг 6. Вердикты и агрегаты ... 6
- Шаг 7. Эксперимент по дооснащению (необязательно) ... 6

4. Как читать метрики ... 7
5. Воспроизводимость ... 8
6. Дообучение ... 9
- 6.1. Переобучение на свежих данных ... 9
- 6.2. Обучение на решениях диспетчеров ... 9

7. Обновление модели на стенде ... 10
- Шаг 1. Обучите и проверьте ... 10
- Шаг 2. Поднимите версию модели ... 10
- Шаг 3. Пересчитайте вердикты и агрегаты ... 10
- Шаг 4. Зафиксируйте в репозитории ... 10
- Шаг 5. Обновите стенд ... 10
- Шаг 6. Проверьте ... 10
- Откат ... 10

8. Чего делать не надо ... 12

1. Что здесь обучается

В проекте четыре сценария, но обучаемая модель одна.

Сценарий	Метод	Требует обучения
M1 отказ датчика	RandomForest с настоящей разметкой	**да**
M2 подтопление	аномалия частоты пусков к личной норме	нет
M3 задымление	многофакторный детектор с отсевом прогонов ТО	нет
M4 проникновение	правила по режиму охраны и мультисенсорности	нет

Такое разделение продиктовано данными, а не выбором: разметки инцидентов в журнале нет, и организаторы это подтвердили. Целевая переменная есть только у отказов — она выводится из семантики значений канала.

Поэтому «обучение модели» в этом документе означает **M1**. Детекторы M2–M4 пересчитываются вместе с витринами и отдельного обучения не требуют.

Обученная модель лежит в репозитории: `models/m1_failure.pkl`, 25,8 МБ. Она нужна, чтобы пересчитать вердикты и воспроизвести метрики, а собрать её заново можно только имея на руках датасет на 16 ГБ.

2. Что потребуется

Ресурс	Требование
Датасет	16 ГБ распакованных CSV в каталоге `dataset/`
Оперативная память	16 ГБ (конвейер написан под этот предел)
Место на диске	20 ГБ под Parquet и витрины
Python	3.12

Docker для обучения **не нужен** — это офлайн-шаг на машине с данными. Лимит памяти DuckDB задан в коде (`core/config.py`), тяжёлые операции сбрасываются на диск. Весь конвейер проходит на ноутбуке; так он и разрабатывался.

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
unzip dataset.zip -d dataset
```

3. Обучение с нуля

Шесть шагов. Замеры — с машины разработки, 16 ГБ ОЗУ.

Шаг 1. Журнал в Parquet

```
python -m scripts.00_import_journal
```

Восемь годовых выгрузок переводятся в колоночный формат: 16 ГБ текста превращаются в 1,9 ГБ Parquet. Ожидаемый вывод:

```
  2019:   20 993 328 строк  →    133 МБ  (2 с)
  ...
всего 313 546 016 событий
```

Что проверить: итог должен быть 313 546 016 событий. Другое число означает, что распакован не тот датасет или выгрузка неполная.

Шаг 2. Витрины

```
python -m scripts.01_build_marts
```

Около **11 секунд** на все витрины. Здесь же срабатывает защита от старой ловушки: в справочнике каналов есть переводы строк внутри значений, и наивное чтение даёт 11 517 строк вместо 11 485. Скрипт падает с внятным сообщением, если число не сошлось.

Что проверить: `dim_channel` — 11 485 каналов, `fact_channel_day` — 4 273 083 строки.

Шаг 3. Тёзки и логические домены

```
python -m scripts.02_twins_and_domains
```

Поведенческий тест каналов с одинаковыми именами и таблица доменов подтверждения.

Что проверить: 1 036 каналов-тёзок в 412 группах, 3 759 доменов.

Шаг 4. Обучение

```
python -m scripts.03_train_m1
```

Меньше минуты. Скрипт печатает размеры выборок, метрики на отложенном периоде, достижимость целевых значений ТЗ, важность признаков и — обязательно — проверку точности объяснения.

Что проверить в выводе:

```
обучающая:     760 000 строк, позитивов 7 500
отложенная:  4 520 313 строк, позитивов 2 401
период обучения 2022-01-01..2024-12-31, проверки 2025-01-01..2026-06-30

  сумма вкладов равна предсказанию: да   (максимальное расхождение 9.99e-16)
```

Последняя строка важнее метрик. Если она говорит «НЕТ», объяснения в карточках расходятся с решением модели, и такую модель ставить нельзя ни при каких метриках.

На выходе: `models/m1_failure.pkl` и `models/m1_metrics.json`.

Шаг 5. Кривая по горизонтам (необязательно)

```
python -m scripts.04_horizon_sweep
```

Обучает ту же модель на горизонтах 24 часа, 3, 7 и 14 суток. Нужна, чтобы показать: целевая точность ТЗ достигается, но на другом горизонте. Четыре обучения подряд, несколько минут.

Шаг 6. Вердикты и агрегаты

```
python -m scripts.05_generate_verdicts   # вердикты → deploy/seed_verdicts.json
python -m scripts.10_recall_math         # цена полноты → models/recall_math.json
python -m scripts.09_build_analytics     # агрегаты диаграмм → deploy/seed_analytics.json
```

Первый скрипт считает вердикты по всем четырём сценариям на последние сутки данных и складывает их в компактный JSON — именно он уезжает в образ. Второй меряет, сколько предупреждений в сутки требует заданная полнота. Третий собирает величины для диаграмм.

Порядок здесь значим. Сборка агрегатов читает готовые файлы кривой горизонтов и цены полноты, поэтому её запускают последней. Если поменять местами, диаграммы покажут прежние числа при новой модели — и это не будет видно ни по какой ошибке.

Что проверить: 162 вердикта, из них 120 отказов, 20 задымлений, 13 проникновений, 9 подтоплений; 13 мультикарточек по объектам.

Шаг 7. Эксперимент по дооснащению (необязательно)

Отдельная цепочка, отвечающая на вопрос «какие датчики поставить». Нужна, только если пересобирается страница «Точность». Подробное описание — в документе «Повышение точности прогноза».

```
python -m scripts.11_calibrate_sensors    # распределения с реальных данных
python -m scripts.14_experiment_controls  # контроли честности метрики
python -m scripts.12_sensor_experiment    # сетка конфигураций
python -m scripts.13_experiment_verdicts  # вердикты синтетического объекта
python -m scripts.15_sync_glossary        # словарь терминов в документ
python -m scripts.16_sync_experiment_tables  # таблица конфигураций в документы
```

Порядок значим и здесь. Генератор синтетического объекта берёт распределения из `models/sensor_calibration.json`, поэтому калибровка идёт первой. Вердикты берут конфигурацию, которую назвал эксперимент, поэтому идут последними.

По времени это самый долгий шаг во всей инструкции, хотя уже не такой долгий, каким был. Сетка — десять конфигураций по три прогона, 400 пикетов, 540 суток; версия с восемью типами датчиков и опросом раз в четверть часа даёт около девятнадцати миллионов записей журнала на прогон.

Первая реализация ставила журнал в базу через `executemany` и тратила на эту сетку около трёх часов. Измерено, где уходило время: миллион строк через `executemany` — 94,7 секунды, тот же миллион через DataFrame — 0,7 секунды, то есть в 129 раз быстрее. После замены на порционную вставку кадрами по два миллиона строк сетка считается за десятки минут, а не за часы. Метрики при этом совпали до последней цифры — проверено сравнением прогонов.

Пересчитать одну конфигурацию, не трогая остальные, можно так:

```
ONLY_CONFIG="плацебо: датчик движения" python -m scripts.12_sensor_experiment
```

Результат сливается в существующий файл, а не перезаписывает его, и встаёт на своё место в сетке. Два таких пересчёта одновременно запускать нельзя: слияние читает и перезаписывает один и тот же файл.

4. Как читать метрики

Эталонные значения поставляемой модели — в `models/m1_metrics.json`:

Метрика	Значение	Как понимать
Позитивов в проверке	2 401	отказов на отложенном периоде
Базовая частота	0,053 %	один отказ на 1 900 канало-суток
PR-AUC	0,0109	главная метрика при таком дисбалансе
ROC-AUC	0,807	здесь почти бесполезна, см. ниже
Precision@50	0,14	
Precision@100	**0,16**	в 301 раз плотнее случайного выбора

ROC-AUC не показатель. При доле позитивов 0,05 % он выглядит прилично и у бесполезной модели. Смотреть нужно на PR-AUC и на precision@k.

Precision@k считается по всей отложенной выборке, а не за сутки. Precision@100 = 0,16 означает: среди ста самых уверенных прогнозов за восемнадцать месяцев шестнадцать оказались настоящими отказами. Это один прогноз в пять суток, а не сто в день. Пересчёт в суточную нагрузку — в разделе 6.4 пояснительной записки.

Что считать поводом насторожиться:

- precision@100 ниже 0,08 — модель хуже, чем была; проверьте, не изменился ли период обучения и не попали ли в выборку массовые базо-дни;
- PR-AUC близок к базовой частоте — модель не лучше случайной;
- проверка аддитивности не прошла — брак, ставить нельзя.

5. Воспроизводимость

Одинаковый запуск даёт байт в байт одинаковую модель. Это проверяется просто:

```
python -m scripts.03_train_m1 && cp models/m1_failure.pkl /tmp/run1.pkl
python -m scripts.03_train_m1 && cmp /tmp/run1.pkl models/m1_failure.pkl
```

Так было не всегда, и эту историю стоит знать. Сначала выборка бралась запросом без сортировки: DuckDB читает параллельно и порядок строк между прогонами не сохраняет, а лес берёт бутстрап-выборки в порядке поступления строк. Итог: **одно и то же обучение с одним и тем же случайным зерном давало разные модели** — два прогона подряд дали precision@100 0,16 и 0,09.

Лечится одной строкой — явным `ORDER BY channel_id, d` в запросе выборки. Стоит это секунд сортировки и снимает вопрос целиком.

Оговорка о поставляемой модели: она обучена **до** этого исправления, поэтому повторное обучение даст другую модель — сопоставимую по качеству, но не ту же самую. Начиная с этой версии кода любая новая модель воспроизводится точно.

6. Дообучение

Под этим словом обычно имеют в виду две разные вещи.

6.1. Переобучение на свежих данных

То, что доступно сегодня. Появилась новая выгрузка журнала — сдвигаем границы периодов в `core/model.py`:

```
TRAIN_FROM, TRAIN_TO = "2022-01-01", "2024-12-31"
TEST_FROM,  TEST_TO  = "2025-01-01", "2026-06-30"
```

и проходим шаги 1–4 заново. Правило одно и нарушать его нельзя: **проверочный период всегда позже обучающего**. Случайное разбиение здесь недопустимо — соседние сутки одного канала почти одинаковы, и модель запомнит канал вместо того, чтобы выучить деградацию. Метрики при этом вырастут, а на реальной работе это никак не скажется.

6.2. Обучение на решениях диспетчеров

То, ради чего в сервисе ведётся журнал решений, — и чего в конвейере пока нет. Скажем прямо: **таблица `decision` сегодня в обучении не используется**. Разметка берётся из данных, а решения накапливаются в оперативном контуре и ждут своего часа.

Почему так: чтобы учиться на отметках «ложное срабатывание» и «неисправность подтверждена», их нужны тысячи, а на новом стенде их ноль. Пары «вердикт — решение» начинают копиться с первого дня эксплуатации; через несколько месяцев работы района их хватит, чтобы перевести детекторы М2–М4 с правил на обучение — и это единственный способ получить разметку инцидентов, которой сегодня нет ни у нас, ни у заказчика.

Что для этого уже готово: справочник причин с флагом «ложное», фиксация решения с автором и временем, журнал прогнозов, связь вердикта с решением. Чего не хватает: собственно шага обучения, который читал бы `decision` и строил по ней целевую переменную.

7. Обновление модели на стенде

Порядок, который не оставляет стенд в несогласованном состоянии.

Шаг 1. Обучите и проверьте

```
python -m scripts.03_train_m1
```

Сравните метрики с прежними (`models/m1_metrics.json` в git). Если precision@100 заметно просел — разберитесь до того, как обновлять стенд.

Шаг 2. Поднимите версию модели

В `core/scoring.py`:

```
MODEL_VERSION = "m1-failure-v2"     # было v1
```

Версия пишется в каждый вердикт. Без этого через месяц будет невозможно понять, какой моделью посчитан конкретный прогноз, — а именно этот вопрос задают, когда прогноз не сбылся.

Шаг 3. Пересчитайте вердикты и агрегаты

```
python -m scripts.05_generate_verdicts
python -m scripts.10_recall_math
python -m scripts.09_build_analytics
```

Шаг 4. Зафиксируйте в репозитории

```
git add models/ deploy/seed_verdicts.json deploy/seed_analytics.json core/scoring.py
git commit -m "model: переобучение на данных по <дату>"
git push
```

Модель, метрики и вердикты должны попадать в один коммит. Разъехавшись, они превращаются в загадку: вердикты одной модели, метрики другой.

Шаг 5. Обновите стенд

```
deploy/remote.sh deploy
deploy/remote.sh reset
```

Второй командой обязательно: загрузка вердиктов идемпотентна и при непустой таблице просто пропускается. Без сброса на стенде останутся старые вердикты при новом коде — и это не будет видно.

Шаг 6. Проверьте

```
curl 'http://localhost:8080/api/verdicts?limit=1' | grep model_version
```

Должна быть новая версия — поле отдаётся в каждом вердикте именно для такой проверки. Дополнительно — 162 вердикта в ленте и диаграммы на странице аналитики, построенные на новых величинах.

Откат

Модель, метрики и вердикты лежат в git одним коммитом, поэтому откат — это возврат файлов и повторное развёртывание:

```
git checkout <коммит> -- models/ deploy/seed_verdicts.json deploy/seed_analytics.json
deploy/remote.sh deploy && deploy/remote.sh reset
```

8. Чего делать не надо

Соблазн	Почему нельзя
Разбить выборку случайно	соседние сутки канала почти одинаковы; модель запомнит канал, метрики вырастут, польза — нет
Убрать маскирование массовых базо-дней	86 % отказных канало-суток приходится на работы бригады; модель выучит график обходов
Ориентироваться на ROC-AUC	при доле позитивов 0,05 % он хорош и у бесполезной модели
Заполнить пропуски нулями	«нормы ещё нет» и «норма равна нулю» — разные состояния; в коде пропуск заполняется значением −1, невозможным для отношения
Увеличить долю позитивов ради красивых метрик	прореживание меняет абсолютную вероятность; ранговые метрики честны, абсолютная — нет
Поставить модель, не прошедшую проверку аддитивности	карточки будут объяснять не то решение, которое принято